THE EVEREST BISTRO

Comprehensive System Specification & Architectural Blueprint

An End-to-End Operational Documentation covering System Roles, QR Security Logic, State Machine Lifecycles, High-Usability User Interfaces, and Real-Time Service Workflows.

Document Version: v3.0 (Master Release)

Status: Approved Architecture

Primary Focus: System Flow & Security Specs

Scope: Full System (Excl. DB Schema)

1. Executive Summary & Project Vision

1.1 Project Vision

The Everest Bistro system is designed to revolutionize the dining experience through a frictionless, high-performance contactless ordering and management web application. The platform serves three distinct operational stakeholders: **Customers** dining at physical tables, **Kitchen Staff** preparing meals in real time, and **Restaurant Owners/Management** controlling operations, point-of-sale (POS) billing, and daily revenue analytics.

The core vision balances ultimate usability for guests (zero app downloads, zero Wi-Fi connection barriers, and simple non-cluttered menu interfaces) with bulletproof operational control for restaurant management.

1.2 Problem Statement & Critical Design Decisions

Traditional QR ordering applications frequently fail due to two major flaws: operational friction and security vulnerabilities.

- **The Local Wi-Fi Friction Trap:** Systems restricted strictly to local area networks (LAN) force customers to ask for Wi-Fi passwords, navigate captive portals, or troubleshoot connection drops. From a Human-Computer Interaction (HCI) perspective, this creates immediate friction that reduces customer satisfaction. *Decision: The system is deployed as a cloud-hosted web application accessible over cellular data or guest Wi-Fi seamlessly.*
- **The Table Hijacking Vulnerability (IDOR):** Naive QR implementations encode plain sequential numbers into browser URLs (e.g., `/menu?table=4`). Curiosity or malicious intent allows users to manually edit the URL parameter to `table=5`, viewing or messing with another guest's active bill. *Decision: The system utilizes static, cryptographically random hashes per physical table paired with dynamic server-issued JWT tokens.*
- **The QR Re-Printing Logistical Nightmare:** Changing QR codes per customer session or printing temporary receipts forces staff to constantly replace physical stickers. *Decision: Physical QR stickers on tables are permanent. The backend dynamically manages session creation and destruction behind the scenes.*

> **Core Principle: Zero Customer Friction**
>
> When a customer sits down, scanning the table QR must open the interactive menu in under 2 seconds without requiring user registration, email input, or app installation. Security and session validation occur invisibly via background handshake protocol.

2. High-Level Technology Architecture

2.1 Component Breakdown & Rationale

The application is structured as a modern, decoupled architecture designed for high availability, real-time synchronization, and low latency.

Layer	Technology Selected	Architectural Rationale & Utility
Frontend Framework	Next.js (React)	Provides ultra-fast client-side routing, mobile-first responsive interfaces, sticky navigation components, and server-side rendering capability. Enables strict route isolation between customer views (`/menu`), kitchen screens (`/kitchen`), and admin consoles (`/admin`).
Backend REST & Sockets	FastAPI (Python)	Delivers asynchronous REST API processing for high-concurrency peak dining hours. Native WebSocket support allows instantaneous bi-directional order broadcasting to kitchen tablets without page polling.
Database & Authentication	Supabase (PostgreSQL)	Offers robust, relational data storage, built-in connection pooling (Bouncer) for sudden traffic spikes, role-based access control, and native storage buckets for menu item photography.
Image Storage Engine	Supabase Storage CDN	Stores high-resolution food and beverage photography. The frontend incorporates automatic fallback logic to display a branded default graphic if an item photo is omitted.
Security & Auth State	HttpOnly JWT Cookies	Encapsulates session state inside encrypted, HttpOnly cookies. Prevents cross-site scripting (XSS) access to tokens and seals the session state strictly to the validated customer browser.

2.2 Network Topology & Communication Protocol

All customer interaction flows over standard HTTPS/WSS (WebSockets Secure) protocols. When an order is placed on a mobile device, Next.js transmits an authenticated JSON payload to FastAPI. FastAPI writes the transaction into PostgreSQL and instantly broadcasts an event to the Kitchen Panel via active WebSockets.

3. Detailed Panel & Screen Specifications

3.1 Customer Panel (Mobile-First View)

Designed exclusively for mobile smartphone viewports. The user interface prioritizes clarity, large touch targets, fast scrolling, and minimal text entry.

Screen / View	UI Components & Elements	Behavioral & Functional Logic
1. Category Menu View	• Sticky Header with Restaurant Name. • Dynamic Category Pill Bar (All, Food, Drinks, Starters, Desserts). • Item Cards: Photo thumbnail, title, price, "Out of Stock" badge, "Add" button. • Bottom Floating Cart Summary Bar.	• **No Search Bar:** Intentionally omitted to reduce cognitive load and simplify interface navigation. • Tapping category pills smooth-scrolls to the corresponding menu section. • Missing photos automatically fall back to standard placeholder image. • Disabled items ("Out of Stock") show greyed out with unclickable buttons.
2. Cart & Checkout Drawer	• Itemized List of selected items with quantity modifiers (+/-). • Subtotal and final amount due. • Prominent **"Confirm & Send Order"** primary button.	• Allows customer to review selections before dispatching to kitchen. • Quantities can be adjusted or deleted before final submission. • Tapping submit locks the selection and sends it directly to FastAPI.
3. Real-Time Order History	• Running list of all items ordered during the session. • Dual-Status Sections: **"Pending Preparation"** vs. **"Delivered"**. • Accumulated Session Total Bill.	• Updates live via WebSocket or polling as kitchen marks items delivered. • Displays accurate real-time bill breakdown. • Refreshes immediately if Admin removes a mistaken or out-of-stock item.

3.2 Kitchen Operational Panel (Tablet / Desktop View)

Designed for touch-screen wall-mounted tablets or monitors in the kitchen area. Requires dedicated staff login credentials.

- **Live Order Ticket Queue:** Orders appear as high-contrast tickets sorted chronologically by submission time. Each ticket clearly highlights the Table Identifier, Item Names, Quantities, and Elapsed Time.

- **Fulfillment Controls:** Large primary **[Mark Delivered]** button on each item or full ticket to signal the food has left the kitchen.
- **Stock Alert Mechanism:** An accessible **[Flag Out of Stock]** button per item allowing kitchen staff to immediately notify management when an ingredient runs out.

3.3 Admin / Owner Panel (Suite View)

A comprehensive management console matching the aesthetic of sleek SaaS dashboards (inspired by modern minimalist administrative suites).

- **Table Operations Dashboard:** Visual grid representing all physical tables. Color-coded status indicators: Green = Empty/Available; Red = Occupied with Active Session UUID. Tapping an occupied table opens its live bill and POS control drawer.
- **POS & Cash Checkout Module:** Calculates final total due. Features a rapid-entry field for "Cash Received," automatically calculates exact "Change Due," and presents a **[Complete Checkout & Close Table]** action button.
- **Dynamic Menu Manager:** Admin UI allowing full control over menu categories, sub-domains, prices, description text, photo uploads, and visibility toggles (Visibility On/Off and Out-of-Stock toggle).
- **Sales Analytics Dashboard:** Live display of total daily revenue accrued across all completed checkouts with date-filtering capabilities for past performance auditing.

4. Comprehensive Functional Requirements (FR)

Req ID	Module / Domain	Detailed Functional Specification
FR-01	Dynamic Catalog Structure	The system must allow the Admin to dynamically create, rename, reorder, and delete parent categories (e.g., "Starters", "Mains") and sub-domains (e.g., "Soft Drinks", "Hot Beverages").
FR-02	Menu Item Management	The system must allow the Admin to add items, assign them to sub-categories, define pricing, edit descriptions, and upload JPEG/PNG photos. If no photo is uploaded, the customer interface must render a standard branded placeholder image.
FR-03	Isolated Panel Login	The system must enforce strict role-based authentication with independent login portals for Kitchen Staff and Admin/Owner accounts. Kitchen accounts cannot access POS or sales data.
FR-04	Temporary Stock Bans	The system must enable the Admin or Kitchen to toggle any menu item to "Out of Stock." This action immediately disables ordering for that item on all customer mobile views in real time.
FR-05	Bill & Order Correction	The system must allow the Admin to intervene in an active session and delete specific ordered items (e.g., wrong order, unavailable dish). Deleting an item must immediately recalculate the running total bill and notify the customer UI.
FR-06	Real-Time Kitchen Dispatch	The system must transmit customer order payloads via WebSockets directly to the Kitchen Panel within 1 second of submission, triggering an audible alert or visual flash without requiring manual page refresh.
FR-07	POS & Change Calculation	The Admin POS screen must display the table's itemized bill, allow entry of cash tendered, compute exact cash change to return to customer, and finalize the payment transaction.
FR-08	Daily Revenue Dashboard	The system must maintain an aggregated sales calculation engine on the Admin Dashboard displaying total monetary sales generated during the current day or for selected historical dates.
FR-09	Immutable Order Archiving	Upon POS checkout completion, the system must retain order records in a `completed` status, saving a immutable `price_at_time` snapshot

Req ID	Module / Domain	Detailed Functional Specification
		for each item to preserve historical accuracy regardless of future price changes.
FR-10	Session Continuity Sync	If a customer closes their mobile browser and rescans the table QR sticker, the system must validate their existing session and restore their exact active order history and cart state.
FR-11	Shared Table Multi-Device Sync	When multiple guests at the same table scan the same QR code during an active session, they must be attached to the same table order session, allowing consolidated billing.

5. Non-Functional & Security Specifications

5.1 Security Architecture: IDOR Prevention & QR Token Handshake

Preventing unauthorized table access or remote bill tampering is achieved through a multi-tiered security handshake protocol combining static hash resolution, server-side session lookup, and JWT cookie issuance.

> **The Cryptographic Static-Hash Paradigm**
>
> Physical QR stickers printed on tables encode an unpredictable static token rather than a readable table number.
>
> - **Bad Practice (Insecure):** `theeverestbistro.com/scan?table_id=4` → Trivially guessable by editing the integer parameter.
> - **Best Practice (Implemented):** `theeverestbistro.com/scan/8f92a7b1-c4d5-4e3a-9b1c` → A 128-bit cryptographically random hash bound to Table 4 in the database.
>
> An attacker scanning Table 4 cannot guess Table 5's token (`3a2b1c4d-9e8f-7a6b-5c4d`), rendering brute-force table hopping mathematically impossible.

5.2 JWT Cookie Lifecycle & Revocation

1. **Scan Handshake:** Customer scans the static QR. FastAPI receives the static hash, checks if the table is set to `OCCUPIED` by staff, and generates a fresh, transient `session_uuid`.
2. **Token Issuance:** FastAPI responds by setting an encrypted `HttpOnly`, `SameSite=Strict` JWT cookie containing the `session_uuid` and redirects the browser to `/menu`.
3. **Request Authorization:** Every order placement or history query sends this automatic cookie. The server validates that the `session_uuid` is still marked `active` in Supabase.
4. **Session Revocation:** When the Admin completes POS checkout, the server explicitly invalidates the `session_uuid` in the database. If the customer re-opens the app from home, the server rejects the revoked JWT cookie and redirects to an expired session notice.

5.3 Performance & Availability NFRs

- **Sub-Second Menu Render:** The customer menu view must load initial categories and items in under 1.5 seconds over standard 4G mobile networks.

- **High-Concurrency Readiness:** The FastAPI backend must utilize Supabase connection pooling (pgBouncer) to sustain up to 500 concurrent active WebSocket and REST connections during peak Friday night dining hours without connection drops.
- **Zero Data Stale Cache:** Next.js fetch calls for active orders must specify `cache: 'no-store'` to guarantee customers never view outdated bill balances or previously closed sessions from browser cache.

6. Table State Machine & Session Lifecycle

6.1 State Machine Lifecycle Diagram

Every physical table in the restaurant moves through strict state transitions managed by staff actions and database locks.

State	Trigger Action	Database Indicator	Customer Access Result
1. EMPTY (Green)	Default state when table is unassigned.	`status = 'EMPTY'` `session_uuid = NULL`	Scanning static QR displays: *"Table not open yet. Please ask staff for seating."*
2. OCCUPIED (Red)	Admin clicks **"Open Table"** on Admin Suite.	`status = 'OCCUPIED'` `session_uuid = GENERATED`	Scanning static QR issues valid JWT cookie and grants full access to Menu and Ordering.
3. CHECKOUT (Yellow)	Customer requests bill; Admin opens POS drawer.	`status = 'CHECKOUT'` `session_uuid = ACTIVE`	Customer can view final bill in read-only mode; new order placement is locked.
4. CLOSED / RESET	Admin completes payment and clicks **"Complete"**.	`status = 'EMPTY'` `session_uuid = DESTROYED`	Session UUID revoked. JWT invalidated. Table resets back to State 1 (EMPTY).

6.2 Handling Regular Returning Customers & Device Isolation

If a regular customer visits on Monday (seated at Table 4) and returns on Friday (seated at Table 8), browser cookies from Monday must not interfere with Friday's dining session.

- **Monday Session Termination:** When the Admin completed Monday's checkout, Table 4's Monday `session_uuid` was destroyed in the database. Monday's cookie became instantly dead.
- **Friday New Handshake:** When the customer scans Table 8's QR on Friday, FastAPI issues a brand-new JWT with Table 8's Friday `session_uuid`, completely overwriting the old cookie in the browser.

7. Actor Specifications & Primary Use Cases

7.1 Actor Profile Definition

- **Customer (Primary Unauthenticated User):** Interacts exclusively via mobile smartphone browser. Can browse menu, place orders, view item preparation status, and track running bill total. No login account required.
- **Kitchen Staff (Authenticated Operational User):** Interacts via tablet panel. Can view live incoming orders, flag items as delivered, and notify management of stock shortages.
- **Admin / Owner (Superuser Authenticated Manager):** Interacts via desktop or tablet. Has full authority over menu configuration, item prices, table session creation/destruction, order corrections, POS cash checkout, and daily revenue reporting.

7.2 Detailed Use Case Descriptions

Use Case UC-01: Table Seating & Session Handshake

Primary Actor:	Customer & Admin/Owner
Preconditions:	Physical table is cleared and set to `EMPTY` state in Admin Suite.
Main Success Scenario:	1. Admin opens Admin Suite and taps "Open Table 4". 2. Backend generates a fresh `session_uuid` and sets table state to `OCCUPIED`. 3. Customer sits at Table 4 and scans the permanent QR sticker. 4. Backend resolves static token, validates active session, and sets HttpOnly JWT cookie on mobile browser. 5. Mobile browser opens menu view instantly.
Exception Flow:	Customer scans QR before Admin taps "Open Table". System displays polite message asking user to notify staff to open their table session.

Use Case UC-02: Submitting an Order & Live Dispatch

Primary Actor:	Customer & Kitchen Staff
Preconditions:	Customer holds active valid JWT cookie linked to an `OCCUPIED` table session.
	1. Customer selects items (e.g., 2x Burger, 1x Coke) and taps "View Cart". 2. Customer reviews totals and taps "Confirm & Send Order".

Main Success Scenario:	3. FastAPI validates JWT, writes items into database, and broadcasts WebSocket payload to Kitchen Panel. 4. Kitchen Panel flashes and sounds chime; order ticket appears instantly. 5. Customer's screen moves items into "Pending Preparation" history list.

Use Case UC-03: POS Checkout & Session Closure

Primary Actor:	Admin / Owner
Preconditions:	Customer meal is finished; items are recorded under occupied table session.
Main Success Scenario:	1. Admin taps red occupied Table 4 on Admin Suite POS. 2. System displays itemized bill total (e.g., $45.00). 3. Admin receives cash payment from guest (e.g., $50.00) and types $50.00 into "Cash Tendered" field. 4. POS screen calculates change due ($5.00). 5. Admin taps "Complete Checkout". 6. System updates order record to `completed`, saves price snapshots for analytics, revokes session UUID, and resets Table 4 to `EMPTY` (Green).

8. End-to-End Operational Process Flows

Flow A: Complete Customer Dining Lifecycle

Step 1: Session Initiation
Staff greets customer at Table 4 and taps "Open Session" on Admin Dashboard. Table 4 turns Red (`OCCUPIED`).

Step 2: QR Handshake & Menu Loading
Customer scans permanent table QR sticker. System validates static hash token, sets HttpOnly JWT cookie, and renders mobile menu.

Step 3: Cart Selection & Dispatch
Customer browses category sections, adds items to cart, and taps "Confirm & Send Order".

Step 4: Kitchen Preparation & Delivery
Kitchen screen chimes with incoming ticket. Staff prepares food and taps "Mark Delivered". Customer's history tab updates status to "Delivered".

Step 5: POS Checkout & Data Archiving
Customer pays cash at register. Admin inputs amount tendered, hands back change, and clicks "Complete Checkout". Session UUID is destroyed; revenue is logged to Daily Sales Dashboard.

Flow B: Order Correction & Stock Invalidation Workflow

Step 1: Kitchen Shortage Signal
Kitchen realizes Coca-Cola syrup is empty and taps "Flag Out of Stock" on the kitchen tablet screen.

Step 2: Menu Invalidation
FastAPI updates Coca-Cola status to `is_available = false`. All active customer mobile menus grey out Coca-Cola in real time.

Step 3: Bill Adjustment

Admin opens Table 4 active bill, deletes the unavailable Coca-Cola line item. Running bill total decreases immediately on both Admin POS and Customer Mobile History view.

9. Operational Edge Cases & Mitigation Strategies

Edge Case Scenario	Potential Operational Risk	Architectural Mitigation Strategy
1. Phone Battery Dies Mid-Meal	Customer cannot view history or show bill to staff.	Because order state lives on the server (linked to Table 4's session UUID), another guest at the table can scan the QR code to join the same active session, or staff can pull up the bill on the Admin POS instantly.
2. Customer Leaves Without Paying	Unpaid bill left lingering on occupied table state.	Staff taps Table 4 on Admin POS and selects "Flag Walkout / Cancel Session". The session is terminated, items are logged as voided/written-off in analytics, and the table resets to `EMPTY`.
3. Wi-Fi or Cellular Network Drop	Order submission fails or double-posts during momentary lag.	Next.js frontend implements idempotent request tokens per order submit button tap. If network reconnects, duplicate posts are ignored by FastAPI backend.
4. Customer Re-scans QR Repeatedly	Potential creation of duplicate sessions or app crashes.	Backend checks if an active session UUID already exists for the table. If found, it simply re-issues the existing JWT cookie, maintaining seamless continuity.

10. POS & Sales Analytics Engine Specifications

10.1 Point of Sale (POS) Calculation Engine

The POS module in the Admin Suite acts as the final ledger validator before session termination.

- **Bill Consolidation Formula:** $ ext{Total Bill} = \sum (ext{Item Price} imes ext{Quantity}) - ext{Removed/Voided Items}$
- **Change Tendered Formula:** $ ext{Change Return} = ext{Cash Received} - ext{Total Bill}$
- **Validation Rule:** POS will reject completion if $ ext{Cash Received} < ext{Total Bill}$.

10.2 Sales Analytics & Historical Reporting

When an order transitions to `completed` state, its monetary metrics are committed to the analytics data engine.

- **Real-Time Daily Sales:** Sum of all completed order totals where `completed_at` timestamp matches the current calendar date.
- **Itemized Snapshot Audit (FR-09):** Each completed order line item saves a permanent `price_at_time` attribute. If the restaurant increases the price of a Burger from $12.00 to $15.00 next month, past financial audit reports accurately retain the $12.00 historical revenue figure.

> ### Document Sign-off & Next Phase Target
>
> This 10-section Master Specification locks in all functional requirements, security parameters, UI specifications, operational process flows, and edge-case behaviors for **The Everest Bistro** system.
>
> **Next Operational Phase:** Database Schema Design & Normalization (defining relational tables, primary keys, foreign key constraints, and index mappings for PostgreSQL/Supabase).